LAB 7: Fuzzing for Input Validation Bugs

Solar Panel Estimate parser • Hypothesis property-based tests

What I tested

Hypothesis generated arbitrary text and `None` values for each of the three input functions. The starter worked for its example inputs, but the generated edge cases exposed crashes in all three functions.

Function	Found in starter	Fix
`clean_site_name`	None caused an attribute error.	Reject non-strings safely; trim and sanitize text.
`parse_monthly_units`	None and empty or malformed readings caused errors.	Validate the full kWh format; return None for invalid input.
`panels_for_roof`	None, missing dimensions, and malformed sizes caused errors.	Parse dimensions defensively; return 0 for invalid or non-positive sizes.

Reflections

1. What did fuzzing reveal? It found failures in inputs that ordinary example-based checks did not cover: missing values, empty text, and incomplete roof dimensions. Hypothesis reduced failures to small examples that made the cause clear.

2. Why use property-based tests here? The tests describe a broad property—each parser should handle arbitrary user input without raising an exception—so Hypothesis can explore and shrink many edge cases automatically.

3. How were the functions hardened? Each function now checks the input type and format before converting it. Invalid site names return an empty string, invalid unit readings return None, and invalid roof sizes return zero panels. Valid inputs still produce the expected normalized name, unit count, and floor panel count.

Verification: 6 Hypothesis tests passed after the fixes (200 generated examples per property test). The course repository README asks for three reflections; they are included above.

SS1 — Buggy module run

SS1 | Buggy starter: normal example run

```
==== Running clean_site_name ====
Enter the installation site name: north #roof
Site name: North Roof

==== Running parse_monthly_units ====
Enter monthly electricity use (e.g. 320 kWh): 320 kWh
Monthly units: 320

==== Running panels_for_roof ====
Enter roof size as LENGTHft x WIDTHft (e.g. 20ft x 15ft): 20ft x 15ft
Panels that fit: 16
```

SS2 — Failing tests on the buggy module

SS2 | Hypothesis exposes input validation bugs

```
$ python -m pytest -q --tb=short test_solar_panel_estimate_parser.py
FFF...                                                                   [100%]

AssertionError: clean_site_name crashed for None: AttributeError
Failing test case: value=None

AssertionError: parse_monthly_units crashed for '': ValueError
Failing test case: value=''

AssertionError: panels_for_roof crashed for None: AttributeError
Failing test case: value=None

3 failed, 3 passed in 1.52s
```

SS3 — Fixed module run

SS3 | Fixed parser: normal example run

```
==== Running clean_site_name ====
Enter the installation site name: north #roof
Site name: North Roof

==== Running parse_monthly_units ====
Enter monthly electricity use (e.g. 320 kWh): 320 kWh
Monthly units: 320

==== Running panels_for_roof ====
Enter roof size as LENGTHft x WIDTHft (e.g. 20ft x 15ft): 20ft x 15ft
Panels that fit: 16
```

SS4 — Passing tests after fixes

SS4 | Hypothesis tests after the fixes

```
$ python -m pytest -q --tb=short test_solar_panel_estimate_parser.py
......                                                                   [100%]

6 passed in 0.68s
```